Aaron Lim Yan Da

(+65) 8207 7872 | aaronlyd03@gmail.com | Linkedin | Github | Portfolio | EN ZH

EDUCATION

National University of Singapore (NUS) **Aug 2024 – Present**

Bachelor of Computing, Business Artificial Intelligence Systems (Information Systems), Honours

SKILLS

- Languages: Java, Python, JavaScript, TypeScript, HTML/CSS, Rust, Golang, SQL, Linux
- AI/ML: Ollama, Dify, ChromaDB, HuggingFace, spaCy, Pandas, Scikit-learn
- Frameworks: Django, React, Node.js/Express, FastAPI, Axum
- Infra & Data: MongoDB, PostgreSQL, Redis, ChromaDB, Docker, Git, Tailscale, Jira

PROJECTS

Odin - AI Tutoring Platform, AI Engineering **Jul 2026 – Present**

Rust/Axum, React/TypeScript, FastAPI, PostgreSQL, Redis, ChromaDB, Ollama (Qwen3.5:9b), sentence-transformers, Dify, Tailscale, Docker

- **Architecture:** Architected a client-server tutoring platform with a Rust/Axum backend and React/TypeScript frontend, connecting a Mac client to a self-hosted GPU node over Tailscale VPN, with multi-modal ingestion (OCR, voice transcription) and a syllabus tracked as a dependency graph across live exercises.
- **Prompt engineering:** Engineered the tutor's system prompt (persona/precision constraints and a LaTeX-vs-fenced-code-block convention), validated via A/B ablation: isolating the formatting clause flipped fenced-code usage 100% to 0%, with no loss in explanation.
- **Context engineering**: Built multi-source RAG (global knowledge base + per-user context) using sentence-transformer (MiniLM-L6-v2) embeddings for trust-weighted, multi-factor retrieval ranking, over-fetch-then-rerank, state-conditional scope switching (subject vs. global), and tiered context compression (raw to summarized to on-demand recall).
- **Harness engineering**: Enforced deterministic grading separate from probabilistic narration (code decides correctness, the model only narrates), verified via unit testing that grading has zero LLM dependency. Implemented agentic tool-calling (SearXNG, Wolfram Alpha, arXiv) with fail-soft dispatch, achieving 85% tool-selection accuracy across a 40-case live test matrix.

Nucleus, AI Engineering **Aug 2026**

NLP, TF-IDF Information Retrieval, OpenAI API | Python, FastAPI, Pydantic

- *Engineered a two-agent pipeline applying TF-IDF retrieval and LLM content generation to remediate storefront gaps via batch edits, surfacing 18 classified gaps across 8 personas.*
- Implemented a code check verifying LLM claims against specs, catching a fabricated claim live.
- Validated JSON data contracts across the pipeline via 82 tests plus live OpenAI checks.

CineVillage, Full-Stack Software Development **Feb 2026 – Apr 2026**

Node.js, Express.js, MongoDB | React, Vite, EJS

- Architect a full-stack cinema platform with a role-based EJS admin console and React/Vite booking SPA backed by a shared MongoDB model, synchronizing movie data through the OMDb API.
- Prevented race-condition double-booking during concurrent checkout using a MongoDB partial unique index and transactional writes, verified through concurrent-request testing.
- Implemented mocked payment validation, booking finalization and automated QR-code e-ticket delivery for an end-to-end checkout workflow.

EXPERIENCE

IDEMIA Public Security | QA Engineer Intern **May 2026 – Present**

Ollama (Qwen3.5), all-MiniLM-L6-v2, Dify | Python, PostgreSQL/pgvector, Prisma, Docker | React/TypeScript

- Identified a repetitive QA bottleneck causing multi-day updates and after-hours work, independently initiating two internal AI automation projects to replace the manual process.
- Built and evaluated a two-stage LLM PoC with routing to reduce costs, parallel execution to lower latency, and schema validation across 70+ tests before a workflow mismatch halted development.
- Independently engineering a self-hosted LLM tool that decomposes tickets into reusable source-of-truth components, reducing a multi-hour workflow to under 30 minutes in controlled testing.
- Implementing semantic matching by embedding atomic components without chunking to automate linking and reduce cost and latency versus LLM comparison before integration.